Session-Based k-Nearest-Neighbours for Retailrocket

A reproducible, fully classical pipeline for anonymous, short-lived browsing sessions with point-in-time features and an honest comparison against popularity

Patrick Oliveira

October 15, 2026

Abstract

We implement session-based k-nearest-neighbours (session-kNN) recommendation for the Retailrocket e-commerce dataset, in which browsing activity is anonymous and ephemeral: a `visitorid` is a cookie, not a durable account, and most sessions contain only a handful of events. Session-kNN represents the current, partial session as a set of recently viewed items, retrieves the most similar historical sessions from a strictly earlier time window, and lets those neighbours vote on the next item – no gradient descent, no embeddings, no sequence model. Every aggregate feature is computed point-in-time against a strict event-order cut-off, the train/validation/test split is session-level and strictly temporal so no session straddles a boundary, and the served ranking is asserted to equal the ranking that was scored offline. On the held-out test partition, tuned session-kNN attains Recall@20 of 0.3545 against 0.3459 for a tuned popularity baseline, with catalogue coverage of 0.930 versus 0.068 – session-kNN reaches comparable accuracy to the strongest non-personalised baseline while recommending from almost the whole catalogue instead of a narrow set of bestsellers. We also show the baseline remains competitive, and in the single-item "cold session" regime slightly ahead, which is a fair characterisation of the data rather than a result to be tuned away.

Contents

1 Notation **3**

2 Introduction **3**
2.1 Scope and claims . . . 3
2.2 Repository . . . 4

3 Data **4**
3.1 Source . . . 4
3.2 Synthetic fixture . . . 4
3.3 Sessionization . . . 4
3.4 Why anonymous sessions, not visitors . . . 5

4 Feature Engineering **5**
4.1 Point-in-time contracts . . . 5
4.2 Catalogue features . . . 5
4.3 An explicitly ablation-only visitor feature . . . 5
4.4 Rationale . . . 5

5 Methodology **6**
5.1 Session representation . . . 6
5.2 Similarity . . . 6
5.3 Candidate retrieval . . . 7

5.4 Recency weighting . . . 7
5.5 Neighbour selection and scoring . . . 7
5.6 Session-segmentation rule (restated) . . . 7
5.7 Baselines . . . 8

6 Evaluation Protocol **8**
6.1 Temporal, session-level split . . . 8
6.2 Next-item-in-session protocol . . . 8
6.3 Cold-session breakdown . . . 8
6.4 Hyperparameter selection . . . 8
6.5 Training/serving parity . . . 9

7 Results **9**
7.1 Session-kNN versus popularity . . . 9
7.2 The cold-session regime is genuinely harder, and popularity wins it . . . 9
7.3 Ablation: similarity metric and recency decay . . . 9
7.4 Random floor . . . 10

8 Conclusion **10**
8.1 Limitations and future work . . . 10

1 Notation

A *session* S is an ordered sequence of items viewed by one visitor with no inactivity gap larger than 30 minutes between consecutive events (Section 3). We write S interchangeably for the sequence and, when only set membership matters, for the set of its items. $\mathcal{I}$ is the full item catalogue. At prediction time a *query session* is a strict prefix of a session, and the task is to rank $\mathcal{I}$ so that the item that actually comes next is ranked as high as possible. $\mathcal{N}(S)$ denotes the k nearest neighbour sessions of a query S, drawn from a pool of historical, completed sessions. Δt denotes elapsed time between two session start times, and λ a decay rate.

2 Introduction

Session-based recommendation differs from classical collaborative filtering in a way that is easy to state and easy to build the wrong system around: there is no stable, long-lived user. Retailrocket's `visitorid` is a browser cookie – it can be cleared, shared across devices, or reassigned, and the typical session contains only a handful of events before the visitor leaves. A model that spends its capacity learning a per-user profile is solving a problem that barely exists in this data; the signal that does exist is almost entirely in *what the visitor has just looked at*.

Session-based k-nearest-neighbours (session-kNN) takes that observation literally. The current, partial session is represented as a set of recently viewed items. At prediction time, the k most similar *past* sessions are retrieved, and candidate items are scored by a similarity- and recency-weighted vote from those neighbours (Section 5). Jannach and Ludewig [3] and Ludewig and Jannach [4] show this kind of neighbourhood method is a consistently strong, often underrated baseline against recurrent neural session models such as GRU4Rec [2] – a session-specific echo of the broader finding that tuned classical baselines frequently match or beat untuned neural recommenders [1]. This project is deliberately scoped to the classical side of that comparison: no neural network appears anywhere in the pipeline.

2.1 Scope and claims

This report accompanies a working repository rather than a novel method. The claims we make are:

1. The pipeline is leakage-free. Session-derived aggregates (session length so far, elapsed time, item popularity) are computed point-in-time, enforced by tests that inject a future event and assert no earlier feature value changes (Section 4).

2. The split is session-level and strictly temporal. No session's events are split across train and evaluation, and neighbour sessions for kNN are drawn only from the training partition, which chronologically precedes every evaluated session (Section 6).

3. The reported numbers are honest. Hyperparameters for every model, baseline included, were selected on a validation partition; the test partition was read once (Section 6).

4. The served ranking equals the evaluated ranking. Session-kNN scores on-the-fly in both the evaluation harness and the Lambda handler, and a parity test asserts the two agree exactly given the same query session and timestamp.

We do *not* claim session-kNN dominates every slice of the evaluation. Section 7 shows the popularity baseline is slightly ahead of it in the hardest, single-item "cold session" regime, and we report that rather than tuning it away.

2.2 Repository

The implementation, infrastructure, and the scripts that generate every table in this document are available in the accompanying repository. The results table (Table 2) is emitted by `scripts/export_results_table.py` and the feature catalogue (Table 1) by `scripts/export_feature_tables` so the document cannot drift away from the code.

3 Data

3.1 Source

The Retailrocket e-commerce dataset [5] is an anonymised log of browsing behaviour on a real online store: roughly 2.7M events from about 1.4M visitors over four and a half months, each row carrying a `timestamp`, `visitorid`, `event` (`view`, `addtocart` or `transaction`), `itemid`, and an optional `transactionid`. A companion `category_tree.csv` and an item-properties changelog (`timestamp, itemid, property, value`) supply a shallow category hierarchy and catalogue metadata such as availability. The raw files are never committed to the repository or downloaded automatically; `scripts/download_data.py -kaggle` fetches them given `KAGGLE_USERNAME`/`KAGGLE_KEY`.

3.2 Synthetic fixture

By default the repository runs against a synthetic fixture generated by the same script with no Kaggle account. The generator is not a schema stub: it reproduces the structural properties the model depends on.

- **Bursty sessions, long idle gaps.** Each visitor is assigned a handful of browsing sessions spread across a configurable time span. Intra-session gaps are seconds to a few minutes; inter-session gaps are drawn independently across the whole span, so they almost always exceed the 30-minute sessionization threshold (Section 3.3) and the segmentation rule is exercised meaningfully rather than trivially.
- **Long-tailed item popularity.** Item weights are drawn from a Pareto distribution, so a small set of items accounts for a large share of views – the regime in which a popularity baseline is genuinely competitive, not a strawman.
- **Latent taste clusters.** Visitors and items are each assigned to one of six latent clusters; a visitor's items are drawn with higher weight from their own cluster. Without this, no similarity-based method could recover structure above chance.
- **Realistic event funnel.** Within a session, an item can progress `view` → `addtocart` → `transaction` with cluster-dependent conversion probabilities, matching the funnel shape of the real dataset.

3.3 Sessionization

Retailrocket carries no explicit session identifier, so one is constructed directly from the inter-event gap. Given a visitor's events sorted by `timestamp`, a new session starts whenever the gap since the previous event exceeds a threshold τ:

$$\text{new_session}(e_t) = \mathbb{1}[\, t_e - t_{e-1} > \tau \,], \qquad \tau = 30 \text{ minutes}, \tag{1}$$

applied independently per visitor (the first event of every visitor also starts a new session). This 30-minute inactivity rule is the convention used throughout the session-based recommendation literature [3, 4] and is what turns an unordered event log into the unit session-kNN actually operates on.

3.4 Why anonymous sessions, not visitors

A durable `visitorid` would let a model build a long-run profile, the way the companion BPR and SVD projects in this portfolio do for MovieLens and the Anime Recommendations Database. Retailrocket does not support that assumption: cookies churn, and the useful history for predicting the next click is almost always confined to the session in progress. Section 4 keeps a single, explicitly ablation-only visitor feature to test this claim rather than merely assert it.

4 Feature Engineering

Every feature consumed by the model or reported in this document is declared in `conf/features.yaml` and implemented in exactly one of four modules (Table 1), generated directly from that file by `scripts/export_feature_tables.py`.

4.1 Point-in-time contracts

Two distinct guarantees are needed, because the features fall into two different classes.

Corpus aggregates (`item_popularity_count`, `visitor_prior_session_count`) are fitted once on the training partition and accept an explicit `cutoff_order` argument so a strict `<` comparison against a global event-order index can be asserted directly: an event with `event_order` $\geq$ the cut-off must not change the value, and an event exactly *at* the cut-off belongs to the future, not the past.

Session-local features (`session_length_so_far`, `time_since_session_start`) have no corpus-level fit at all: they are computed per event row from only the *other* events in the same session that occurred strictly before it. `time_since_session_start` in particular reads the timestamp of the most recently *observed* event, not the timestamp of the row being scored – the latter is the quantity the model is trying to predict alongside the next item, and using it would be the single most tempting leak in this codebase.

Context features (`hour_of_day`, `is_weekend`) are deterministic functions of the event being scored and are declared `point_in_time: false`: they carry no leakage risk because the wall-clock time of a prediction request is known by definition, unlike the item that will actually be clicked.

4.2 Catalogue features

`item_category` is read directly from the item-properties changelog, independent of any interaction history, so cold-start items – never seen in training – still carry a category and are not excluded from candidate scoring.

4.3 An explicitly ablation-only visitor feature

`visitor_prior_session_count` counts how many training sessions a `visitorid` started before the current one. The primary session-kNN method never conditions on it: the whole premise of this project is that anonymous, short-lived sessions can be modelled without a persistent identity. The feature exists so that premise is testable (does a returning cookie help at all?) instead of merely asserted; Section 7 reports the answer.

4.4 Rationale

`item_popularity_count` How often an item was viewed in training sessions up to the cut-off. Drives the popularity baseline and the cold-start fallback ordering for items with no neighbour-session support.

Table 1: Engineered features. Aggregates (`fit_on: train_only`) are computed point-in-time against a strict event-order cut-off; session-local features (`fit_on: none`) are point-in-time by construction, since they read only strictly-earlier positions in the same session.

Feature	Family	Fit scope	Point-in-time
`item_popularity_count`	item	`train_only`	✓
`item_category`	item	`catalogue`	–
`visitor_prior_session_count`	user	`train_only`	✓
`session_length_so_far`	interaction	`none`	✓
`time_since_session_start`	interaction	`none`	✓
`hour_of_day`	context	`none`	–
`is_weekend`	context	`none`	–

`item_category` Catalogue attribute from the item-properties changelog. Known before any interaction, so it is available for items that never appear in training and can be used to break ties among equally popular candidates.

`visitor_prior_session_count` Number of training sessions a visitorid has started before the current one. This is an optional, strictly ablation-only signal: visitorid is a cookie, not a durable account, so the primary session-kNN method never depends on it, but a returning cookie is still weak positive evidence.

`session_length_so_far` Number of events already observed in the current session, i.e. the 0-indexed position itself. Short sessions get little context, which is exactly the "cold session" regime broken out in the evaluation report.

`time_since_session_start` Elapsed time between the session's first event and the *last observed* event (not the event being predicted). Captures browsing pace – a visitor who has lingered for ten minutes behaves differently from one three seconds into a session.

`hour_of_day` Hour extracted from the event being scored. Not an aggregate over history, so it carries no leakage risk: the wall-clock time of the prediction request is known by definition.

`is_weekend` Weekday/weekend split of browsing behaviour, used as an ablation feature alongside hour_of_day to test whether coarse temporal context improves candidate re-ranking over neighbour-vote scores alone.

5 Methodology

5.1 Session representation

A query session S (the prefix observed so far) is represented purely as a *set* of item identifiers, discarding order within the session. This is a deliberate simplification: order and timing are reintroduced elsewhere (the session-local features of Section 4, and the recency weight of neighbour *sessions* below), but the core retrieval step compares sessions on co-occurrence alone, which is what makes it classical set arithmetic rather than a sequence model.

5.2 Similarity

Two sessions are compared with cosine similarity over their binary item indicator vectors, which is equivalent to normalizing the raw intersection size by the geometric mean of the two session sizes:

$$\text{sim}(S, N) = \frac{|S \cap N|}{\sqrt{|S|\,|N|}}. \tag{2}$$

This is the primary similarity used throughout the report. As an ablation (Section 7) we also evaluate Jaccard similarity, which normalizes by the union instead of the geometric mean and therefore penalizes a large symmetric-difference more aggressively:

$$\text{sim}(S, N) = \frac{|S \cap N|}{|S \cup N|}. \tag{3}$$

Both are 0 whenever the sessions share no item and 1 when they are identical; cosine is less sensitive to a large size mismatch between S and N, which matters here because sessions range from one to a few dozen events.

5.3 Candidate retrieval

Scoring every training session against every query would cost $O(|\text{train sessions}|)$ per request. Instead, an inverted index maps each item to the training sessions that contain it, and the candidate pool for a query S is the union of sessions indexed by any item in S – any session with zero overlap has sim $= 0$ by Equations 2–3 and can be skipped entirely. A configurable cap retains only the `sample_size` most recent candidates by session start time, bounding per-request cost as the training log grows.

5.4 Recency weighting

Older sessions are discounted exponentially by the elapsed time between the query session's start and the neighbour session's start, Δt (measured in hours):

$$w(N) = e^{-\lambda \Delta t}. \tag{4}$$

$\lambda = 0$ disables decay (a pure similarity vote); larger λ discounts stale sessions faster. Section 7 reports that, on this dataset, the tuned configuration sets $\lambda = 0$ – recency decay is implemented and ablated, but does not improve on this particular fixture, which we report rather than silently dropping the feature.

5.5 Neighbour selection and scoring

The k nearest neighbour sessions, $\mathcal{N}(S)$, are the k candidates with the highest $\text{sim}(S, \cdot)$ (ties broken by recency). Every candidate item is then scored by summing the similarity- and recency-weighted votes of the neighbours that contain it:

$$s(i, S) = \sum_{N \in \mathcal{N}(S)} \text{sim}(S, N)\, w(N)\, \mathbb{1}[\, i \in N \,]. \tag{5}$$

Items already present in the query session are excluded from the candidate set before ranking, so the model is never evaluated on "recommending" what the visitor has already just viewed. There is no parameter being learned by gradient descent anywhere in Equations 2–5: the entire method is counting, set membership, and a partial sort over at most $k \times |\mathcal{I}|$ score contributions.

5.6 Session-segmentation rule (restated)

The unit this method operates over is itself defined classically (Equation 1): a session is the maximal run of a visitor's events with no inter-event gap exceeding 30 minutes. Session-kNN is only as good as this boundary; too short a threshold fragments genuine single sessions, too long merges unrelated visits and dilutes the within-session signal the method relies on.

5.7 Baselines

Popularity. Ranks every candidate by global training-time view count, ignoring the query session entirely. It is the floor that justifies retrieving neighbours at all, and – as Section 7 shows – a genuinely competitive floor on this dataset, not a strawman.

Random. A seeded permutation per query. It calibrates the metric scale and, with full catalogue coverage by construction, demonstrates that coverage alone is not evidence of quality.

6 Evaluation Protocol

6.1 Temporal, session-level split

Sessions are ordered by start time and partitioned by position in that ordering into train (70%), validation (15%) and test (15%). Every event belonging to a session is assigned to that session's partition, so no session is ever split across the boundary – the unsafe alternative, a row-level split, would let a model's feature-fitting step observe part of a session it is later evaluated on. Validation and test sessions shorter than two events are excluded from evaluation entirely (they contribute no next-item target), but remain available in train for fitting corpus aggregates.

Session-kNN's neighbour pool is restricted, by construction of this split, to training sessions only (Section 5). Because every training session's start time precedes every validation/test session's start time, this also means every neighbour a query session can retrieve chronologically precedes it – there is no scenario in which a session "recommends itself" or borrows structure from a session evaluated alongside it.

6.2 Next-item-in-session protocol

For every session in the evaluated partition, the protocol walks every prefix length $p = 1, \ldots, |S|-1$: the first p items become the query session, the item at position p is the held-out target, and the items already seen are excluded from the candidate set. A session of length L contributes $L - 1$ evaluated positions; length-1 sessions contribute none, since they carry no next item.

Recall@K is the fraction of evaluated positions where the held-out target appears anywhere in the top-K list. Because exactly one item is relevant per position, Recall@K here is equivalent to hit rate.

MRR is the mean reciprocal rank of the target when it appears in the top-K list (and 0 when it does not), rewarding the method for ranking the correct item *near the top*, not merely somewhere in the list.

Coverage is the fraction of the catalogue that appears in at least one top-K list across the whole evaluation, measuring whether a method recommends from a broad slice of the catalogue or collapses onto a narrow set of items – the latter is invisible to Recall@K and MRR alone.

6.3 Cold-session breakdown

Positions with $p = 1$ – exactly one observed item – are also reported separately. This is the hardest and, for Retailrocket, the most common regime: short, anonymous sessions provide almost no context before the first recommendation is due. Reporting it separately from the aggregate prevents a method that only wins on longer, easier prefixes from looking uniformly better than it is.

6.4 Hyperparameter selection

Every hyperparameter – session-kNN's k, `sample_size`, λ and similarity metric – was selected by grid search on the *validation* partition (`scripts/export_results_table.py` reads the test

Table 2: Next-item-in-session ranking quality on the held-out test partition ($K = 20$). Hyperparameters were selected on the validation partition; the test partition was read once. The "cold" columns restrict the metric to single-item prefixes.

Model	Recall@20	MRR@20	Coverage	Recall@20 (cold)	N cold
SessionKNN	**0.3545**	0.1701	0.9300	0.4356	489
Popularity	0.3459	0.1490	0.0675	0.4601	489
Random	0.0478	0.0089	1.0000	0.0511	489

partition only after selection, and only once). The baselines have no hyperparameters to tune beyond the shared evaluation cut-off K.

6.5 Training/serving parity

Because session-kNN scores on-the-fly rather than from a precomputed per-entity table (Section 6.5), the parity risk is different from a matrix-factorization project: it is not "did we serve a stale precomputed list" but "does the request-time Lambda reproduce the exact ranking the offline harness scored for the same query session and timestamp". A dedicated test constructs a query session, calls `model.recommend` directly, and separately invokes the Lambda handler with the same items and an explicit `query_start_ts`, and asserts the two rankings are identical.

7 Results

7.1 Session-kNN versus popularity

Tuned session-kNN attains Recall@20 of 0.3545 against 0.3459 for the tuned popularity baseline on the held-out test partition – a modest but real margin, and consistent in direction with MRR@20 (0.1701 versus 0.1490). The difference that matters more for a production system is coverage: 0.930 against 0.068. Popularity necessarily recommends from a small, fixed set of bestsellers regardless of what the visitor is looking at; session-kNN's accuracy is comparable *while* drawing recommendations from almost the entire catalogue, which is the actual argument for using it over a popularity list in a real store, independent of the raw accuracy numbers.

7.2 The cold-session regime is genuinely harder, and popularity wins it

Restricted to single-item prefixes, popularity's Recall@20 is 0.4601 against session-kNN's 0.4356. This is the expected failure mode of a neighbourhood method: with only one observed item, the candidate neighbour pool is small and noisy, and the recency/similarity vote has little to work with. Popularity, by construction, is unaffected by session length. We report this rather than hiding it, because it motivates a concrete production decision: blend session-kNN with the popularity fallback for very short sessions rather than deploying session-kNN unconditionally (Section 6.5 already implements half of this blend as the cold-start path for requests with zero recognised items; extending it to single-item sessions is a one-line threshold change, not a reason to prefer a different method overall).

7.3 Ablation: similarity metric and recency decay

Grid search on the validation partition selected cosine similarity over Jaccard, and $\lambda = 0$ (no recency decay) over every nonzero decay rate tried. The latter is the more surprising finding: Equation 4 is implemented and tested (Section 5), but on this synthetic fixture the taste-cluster structure is stationary across the whole time span, so discounting older sessions only throws away useful neighbours without compensating for any real preference drift. On the real Retailrocket

log, where seasonal and catalogue turnover effects are present, we expect $\lambda > 0$ to earn its place; the ablation is kept in the code specifically so that re-tuning against the real dataset is a one-line change, not a re-implementation.

7.4 Random floor

Random recommendation attains Recall@20 of 0.0478, close to the theoretical floor $K/|\mathcal{I}| = 20/400 = 0.05$ for this fixture, and full catalogue coverage by construction – confirming that coverage alone is not a quality signal, only a necessary complement to Recall@K and MRR.

8 Conclusion

Retailrocket's browsing log has no durable user to personalise against, and most sessions are too short for a sequence model to find much to condition on. Session-based k-nearest-neighbours fits that regime directly: it represents the current session as a set of items, retrieves similar past sessions from a strictly earlier, leakage-safe pool, and lets a similarity- and recency-weighted vote (Equations 2–5) produce the ranking – no embeddings, no gradient descent, no sequence model anywhere in the pipeline.

On the held-out test partition, tuned session-kNN modestly outperforms a tuned popularity baseline on Recall@20 and MRR@20 while covering more than thirteen times as much of the catalogue, and we have been explicit about the one regime – single-item "cold" sessions – where popularity remains competitive rather than presenting an unconditionally favourable picture. Every point-in-time contract is enforced by a failing-by-default test, the split is session-level and strictly temporal, and the serving path is asserted to reproduce the offline ranking exactly rather than assumed to.

8.1 Limitations and future work

- **Synthetic fixture.** The headline numbers in Section 7 are measured on the synthetic fixture, not the real Kaggle dataset, so the repository is runnable with no Kaggle account. `scripts/download_data.py -kaggle` downloads the real log for anyone who wants to reproduce the result against it; we expect the direction of the comparison to hold but not necessarily the exact margins.
- **Item side-information is underused.** `item_category` is engineered and tested but not yet blended into the scoring function; a category-aware fallback could plausibly close some of the cold-session gap reported in Section 7.
- **A single blended serving policy.** The current serving path (Section 6.5) only falls back to popularity for zero-item requests. Extending the blend to short-but-nonzero sessions, as motivated in Section 7, is future work rather than a limitation of the method itself.
- **Candidate truncation.** `sample_size` bounds the neighbour search for scalability; on a catalogue many times larger than this fixture's, a cheaper approximate retrieval step (e.g. locality-sensitive hashing over session vectors) would likely be needed to keep the per-request cost bounded without raising the truncation threshold.

References

[1] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, pages 101–109, 2019.

[2] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016.

[3] Dietmar Jannach and Malte Ludewig. When recurrent neural networks meet the neighborhood for session-based recommendation. In *Proceedings of the 11th ACM Conference on Recommender Systems (RecSys)*, pages 306–310, 2017.

[4] Malte Ludewig and Dietmar Jannach. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 28(4-5):331–390, 2018.

[5] Retailrocket. Retailrocket recommender system dataset. Kaggle, 2017. https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset.